VTU Athletics Management Platform — Product Requirements Document

Version: 1.0
Purpose: This document is written so that a reader who has never seen the app can follow it end-to-end — every architectural decision and every screen, every button, and what happens when it's tapped — without needing to look at the code.

Table of Contents

Section 1 — System Architecture

1. Design Principles
2. Revised High-Level Architecture
3. Module Breakdown
4. Offline-First Scan Sync
5. Photo-Finish Integration
6. Data Architecture (Schema Overview)
7. Real-Time Layer
8. Role & Permission Model (Capability-Based)
9. Deployment & Infrastructure
10. Capacity Plan & Non-Functional Requirements
11. Security

Section 2 — Page-by-Page, Button-by-Button Specification

1. Participant
2. Coach
3. Referee / Official
4. Event Head
5. Hostel Warden
6. Food Official
7. Photo-Finish Operator
8. Admin

9. Super Admin

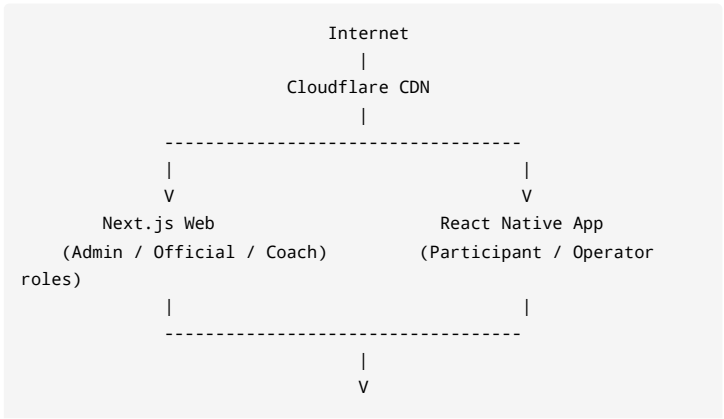
SECTION 1 — SYSTEM ARCHITECTURE

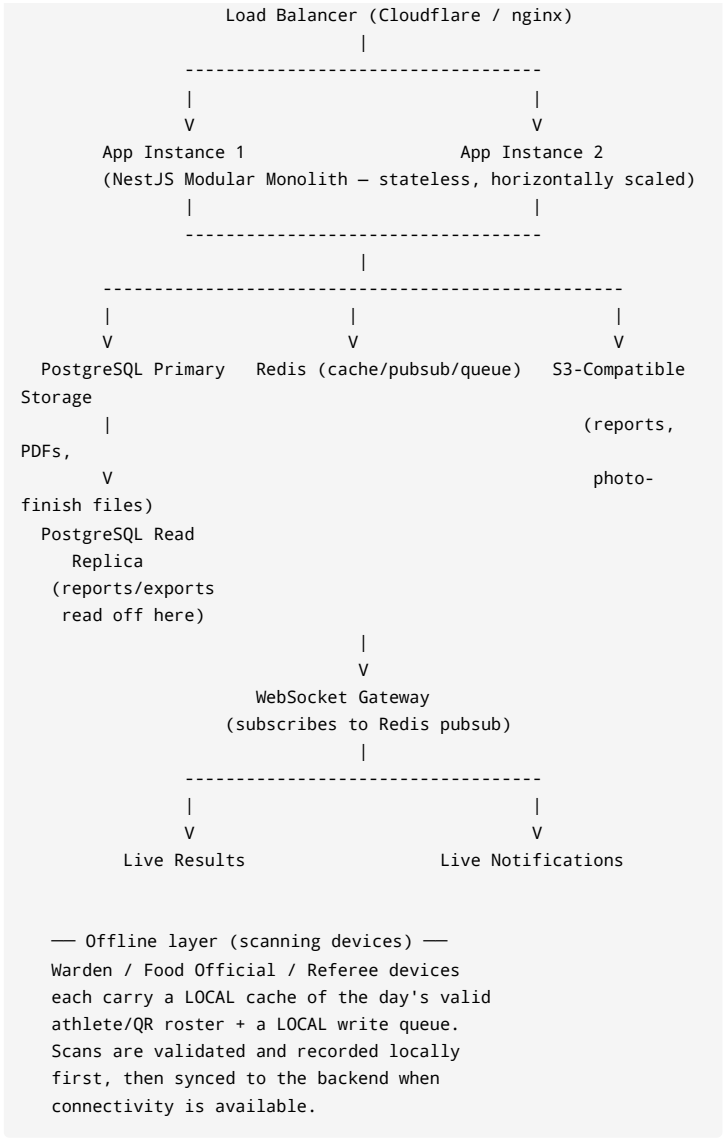
1. Design Principles

This system is designed around what actually breaks event management software, in priority order:

1. **Data durability beats compute redundancy.** You can survive an API instance dying. You cannot survive losing meal/attendance/result records for 500 athletes with no re-run possible.
2. **Assume the network will fail at the worst moment.** Scanning happens outdoors, in crowds, on shared wifi/cellular. Every scan-based flow must work offline and sync later.
3. **One deployable, not five imaginary ones.** At this scale (3,000 users, 3-day event), a modular monolith with clean internal boundaries beats "microservices" that are actually deployed and scaled together anyway.
4. **Every write is idempotent.** The same scan replayed twice (bad network, retry, duplicate tap) must never create two meal records or two check-ins.
5. **Humans override machines.** Admin/Super Admin always have a manual path around any automated flow — QR scanner breaks, someone still needs to get fed.

2. Revised High-Level Architecture





What changed from the original diagram and why:

- Collapsed "5 backend services" into one NestJS app with 9 internal modules. They're still cleanly separated (own folder, own DB tables, talk to each other through defined interfaces) — but you deploy, scale, log, and debug **one thing**,

not five. Split a module into its own service later only if it has a genuinely different scaling profile (the QR/scan module is the most likely candidate if you ever run this at 10x the size).

- Added a **read replica**. Reports/exports (admin dashboards, CSV/PDF generation) hit the replica so they never compete with live scan traffic on the primary.
- Added an explicit **offline layer** — this didn't exist in the original design and is the single highest-risk gap for event day.

3. Module Breakdown

Each module owns its own tables and exposes a clear internal interface. Modules communicate through the module's service class (in-process), not HTTP — that's what makes this a monolith instead of distributed spaghetti.

Module	Responsibility	Key Tables	Talks To
Auth	Login (Phone/Email OTP), JWT issuance/refresh, role & capability checks	`users`, `roles`, `capabilities`, `role_capabilities`	All modules (middleware)
Athlete	Athlete profile, college, events registered, chest number, QR identity	`athletes`, `colleges`, `sports`, `athlete_events`	Auth, QR, Event
QR	Generates and validates signed QR identities; central identity layer	`qr_identities`	Athlete, Attendance, Meal, CallRoom
Event	Events, heats, lane allocation, schedules, call room, qualification logic	`events`, `heats`, `lanes`, `event_status_log`	Athlete, Results, Notification

Module	Responsibility	Key Tables	Talks To
Accommodation	Hostel/room/bed allocation	`hostels`, `rooms`, `bed_assignments`	Athlete, Attendance
Meal	Meal scan validation, prevents duplicate serving	`meal_transactions`	QR, Sync
Attendance	Arrival tracking (left home → in transit → reached → checked-in), hostel check-in/out	`arrival_status`, `hostel_checkins`	QR, Accommodation
Results	Score/time entry, ranking, medal table, photo-finish import	`results`, `medal_table`, `photo_finish_imports`	Event, Notification
Notification	Scheduled + event-triggered push notifications (FCM)	`notifications`, `notification_log`	Event, Results
Sync	Reconciles offline scan queues from devices; conflict detection	`sync_events`, `sync_conflicts`	Meal, Attendance, QR

4. Offline-First Scan Sync

This is the module the original design was missing, and it's the one most likely to cause a visible failure on event day.

Pre-event (each morning, before scanning starts):

- Each warden/food-official/referee device pulls and caches: the day's valid athlete roster, QR signing public key, and current meal/attendance state

relevant to their station.

- Cache is refreshed automatically whenever connectivity is available; last successful sync time is shown on-device so operators know how stale their data is.

At scan time:

1. Device scans QR → validates signature and athlete identity **locally**, against the cached roster (no network call required for validation itself).
2. Device checks local "already consumed" flag (e.g., "breakfast already scanned for this athlete") before allowing the action.
3. The scan event `{athlete_id, action_type, timestamp, device_id, operator_id, idempotency_key}` is written to a **local queue** immediately — the operator sees success/fail instantly regardless of network state.

Background sync:

- Queued events are pushed to the server whenever connectivity exists, with exponential backoff retry.
- The idempotency key (`device_id + athlete_id + action_type + local_timestamp`) means a retried push can never create a duplicate record server-side.

Conflict handling:

- If two devices scan the same athlete for the same meal within a short window (possible if wifi drops mid-morning and two counters both "go offline-first"), the server accepts the **first** valid scan by server-received order and flags the second as a **conflict**, not a duplicate meal — it does not silently drop it.
- Conflicts appear in a queue on the Admin dashboard for manual resolution (e.g., "Athlete 1043 scanned for breakfast at 2 counters — confirm which stands").

Why this matters: without this layer, a single dead access point during peak breakfast scanning stops meal service entirely, or forces staff to wave athletes through unchecked — which is worse than no system at all.

5. Photo-Finish Integration

The original design hand-waved this as "Import Service → Validation → Results DB." That's not a spec — it's a hope. Before writing any code here:

Step 0 (do this first, before anything else in this document): identify the actual timing hardware being used (e.g., FinishLynx, Alge Timing, Omega) and get its real export format — serial feed, timed CSV/XML export, or vendor API. This is unknown-unknown territory and should be the first thing prototyped, not the last thing diagrammed.

Adapter pattern (once the format is known):

```
Photo-Finish Hardware
        |
        V
 Vendor-Specific Adapter   (implements a common PhotoFinishAdapter
interface —
        |                   swap adapters if hardware changes
without touching
        V                   the rest of the pipeline)
 Normalized Result Format
   { event_id, lane, time, sequence }
        |
        V
 Validation
   - lane maps to a real registered athlete for this heat?
   - time is plausible (not 0, not negative, not a duplicate of
last import)?
        |
        V
 Staging Table (photo_finish_imports)
   - visible to Event Head before it goes live
        |
        V
 Event Head Approval  →  Results Table  →  Live Publish
(WebSocket)
```

- Results are **never** auto-published straight from the timing hardware — there's always a human approval step (Event Head) between import and publish, because a mis-mapped lane going live instantly is a much worse failure than a 30-second delay.

6. Data Architecture (Schema Overview)

Core relational schema (PostgreSQL). This is the ~15-table core; exact columns get defined at implementation time, but relationships are fixed here so the rest of the spec is unambiguous.

```
users (1) ──< roles (via role_capabilities) ──< capabilities
athletes (N) ──> colleges
athletes (1) ──< qr_identities
athletes (N) ──< athlete_events >── events
events (1) ──< heats (1) ──< lanes >── athletes
hostels (1) ──< rooms (1) ──< bed_assignments >── athletes
athletes (1) ──< arrival_status (log, append-only)
athletes (1) ──< hostel_checkins (log)
athletes (1) ──< meal_transactions (log)
events (1) ──< results >── athletes
results (1) ──< medal_table
events (1) ──< photo_finish_imports
users (1) ──< notifications (log)
devices (1) ──< sync_events (log) ──< sync_conflicts
```

Design rule: every scan/action table (`meal_transactions`, `hostel_checkins`, `arrival_status`, `sync_events`) is **append-only** — you never update or delete a row, you insert a new one with a timestamp. This means the full history is always reconstructable and nothing is silently overwritten, which matters a lot when there's a dispute ("I did check in, look at the log").

7. Real-Time Layer

```
Referee/Event Head updates a result
                |
                V
        Results Module (validates, writes to DB)
                |
                V
        Redis Pub/Sub  (publish on channel: event:
<event_id>:results)
                |
                V
        WebSocket Gateway  (subscribed per-channel)
                |
                V
        All connected clients watching that event
        (Live Results screen, Coach dashboard, Admin dashboard)
```

Redis Pub/Sub is what lets this scale across the 2 app instances — a result written by the instance handling the Event Head's request gets broadcast to clients connected to *either* instance's WebSocket Gateway.

8. Role & Permission Model (Capability-Based)

The original design hardcodes 9–11 named roles directly into permission checks. That's brittle — the moment someone needs a one-off permission change mid-event (very likely), you're editing code. Instead: roles are just **named bundles of capabilities**, and permission checks are always against a capability, never a role name.

Capabilities (the actual permission units):
`manage_users`, `manage_events`, `scan_operations`, `enter_results`, `approve_results`, `view_reports`, `export_data`, `override_actions`, `manage_hostel_allocation`

Roles → capability bundles (editable at runtime by Super Admin):

Role	Capabilities
Super Admin	all capabilities + `override_actions`
Admin	all except `override_actions`
Event Head	`manage_events`, `enter_results`, `approve_results`, `view_reports` (scoped to their event)
Referee	`scan_operations`, `enter_results` (scoped to their event)
Coach	`view_reports` (scoped to their athletes)
Hostel Warden	`scan_operations` (scoped to hostel check-in/out)
Food Official	`scan_operations` (scoped to meals)
Photo-Finish Operator	`enter_results` (import only, no approve)
Participant	none — read-only own-data views, not capability-gated

This collapses "9 roles" into 9 capabilities that compose — new roles (e.g., a "Track Head" for one specific sport) are just a new bundle, not new code.

9. Deployment & Infrastructure

Layer	Choice	Why
App	2× NestJS instances behind a load balancer	Stateless — horizontal scaling is trivial; redundancy for the least likely failure point
Database	PostgreSQL primary + 1 read replica	Primary takes writes; replica serves reports/exports so they never contend with live scan traffic
Backups	Automated daily full backup + continuous WAL archiving, **restore tested before event day**	This is the actual single point of failure in the original design — untested backups are not backups
Cache/Queue	Redis, AOF persistence enabled	Used for pub/sub and sync queues, not just cache — needs to survive a restart
Object Storage	Cloudflare R2 or S3	Reports, PDFs, photo-finish raw files
Environments	Staging (mirrors prod) + Production	Run a full offline-sync and photo-finish drill on staging before the real event
Mobile	React Native + Expo, single codebase, role-based screen routing	One app, roles determine what renders
Web	Next.js, role-based routing	Admin/Official/Coach primarily use web

10. Capacity Plan & Non-Functional Requirements

The original design named the risk ("500 athletes open the app at 8:30 AM") but never quantified it. Rough targets to actually load-test against before event day:

- **Peak concurrent users:** ~3,000 (full event population), with a burst of ~500 within a 5-minute window at scan/check-in peaks (breakfast, call rooms).
- **WebSocket connections:** up to ~3,000 concurrent during live results windows.
- **DB connection pool:** sized for peak concurrent write bursts from scan sync (not steady-state — steady-state load is low, bursts are the risk).

- **Target:** p95 API response time under 500ms at 2x expected peak load, verified with an actual load test (k6 or similar) before event day, not assumed.

11. Security

- **QR payload:** `{id, event_year, sig}` — HMAC-signed server-side; signing key rotated per event year so old QR codes can't be replayed the following year.
- **Auth:** short-lived JWT (e.g., 1 hour) + refresh token; OTP-based login (phone/email), no passwords to leak.
- **Rate limiting:** scan endpoints and sync endpoints are rate-limited per device to prevent a malfunctioning device from hammering the API.
- **Audit log:** every `override_actions` use (Super Admin force check-in, result override, record deletion) is logged with who/when/what — this is the accountability trail for disputes.

SECTION 2 — PAGE-BY-PAGE, BUTTON-BY-BUTTON SPECIFICATION

Each screen below lists: what the user sees, every interactive element, and exactly what happens when it's used — success path, failure path, and what's written to the database.

---

1. PARTICIPANT

1.1 Login Screen

Purpose: Authenticate the athlete before any other screen is reachable.
Elements: Phone/Email toggle, input field, "Send OTP" button, 6-digit OTP input (appears after code sent), "Verify" button, "Resend OTP" link (disabled 30s countdown).
Buttons:

- **Send OTP** → validates the phone/email format client-side, calls `POST /auth/otp/send`. Success: OTP input appears, 30s resend countdown starts. Failure (unregistered number): "This number isn't registered — check with

your college coordinator," no OTP sent (prevents spamming unregistered numbers).

- **Verify** → calls `POST /auth/otp/verify`. Success: JWT + refresh token stored, role read from response, user routed to Participant Home. Failure (wrong code): inline error, 3 attempts before a 60s lockout. Failure (expired code): "Code expired, request a new one," OTP field clears.
- **Resend OTP** → disabled until countdown hits 0, then re-triggers Send OTP flow.

1.2 Home Screen

Purpose: Landing dashboard — orientation at a glance.
Elements: Header card (name, college, chest number, current status badge — e.g. "Checked-In"), events list, 6 navigation cards (My QR / Schedule / Results / Hostel / Notifications / Maps).
Buttons:

- Each **navigation card** → routes to the corresponding screen (2.3–2.8 below). No API call from the card itself; the destination screen fetches its own data.
- **Status badge tap** → opens a read-only bottom sheet showing the arrival-status timeline (see 1.3) — informational only, doesn't change state from here.

1.3 My QR Screen

Purpose: Single identity used for every scan interaction in the system (hostel, meals, call room, attendance).
Elements: Large QR image, athlete name, athlete ID, chest number, college, "Refresh QR" button, "Download QR" button.
Buttons:

- **Refresh QR** → calls `GET /qr/refresh`. Regenerates the signed payload (same identity, new signature/nonce) — used if a screenshot of the QR leaks and needs invalidating. Old QR image becomes invalid immediately server-side.
- **Download QR** → saves the QR as an image to the device's gallery/files, no network call (renders client-side from already-loaded data).

1.4 Arrival Tracking Screen

Purpose: Self-reported travel status so hostel/food ops can anticipate load.
Elements: 4-step progress tracker (Left Home → In Transit → Reached City →

Checked Into Hostel), one active button matching the current allowed next step.
Buttons:

- **"I've Left Home" / "I'm In Transit" / "I've Reached the City"** (only one is active at a time, matching current state) → calls `POST /attendance/arrival-status` with the new status. Writes an append-only row to `arrival_status`. Cannot go backward — the UI only ever shows the next forward step, previous steps are shown greyed out as history.
- **"Checked Into Hostel"** step is **not tappable by the participant** — it's set automatically when a warden scans their QR at the hostel (see Hostel Warden 5.2), to prevent self-reported false check-ins.

1.5 Hostel Screen

Purpose: Shows allocation and lets the participant view (not perform) check-in/out status.
Elements: Hostel name, room, bed number, warden contact (tap-to-call), embedded map preview, check-in/check-out status with timestamp.
Buttons:

- **Map preview** → opens Google Maps with the hostel location pinned.
- **Warden contact** → opens the device's phone dialer pre-filled with the warden's number.
- No check-in/out button here — that action belongs to the warden's scan (Section 5), this screen is read-only for the participant by design (prevents self-reported hostel fraud).

1.6 Food Screen

Purpose: Shows meal status for the day.
Elements: 4 meal cards (Breakfast/Lunch/Snacks/Dinner), each showing "Consumed at [time]" or "Not yet consumed."
Buttons: None — this screen is read-only; the actual scan action happens at the Food Official's counter (Section 6). Pull-to-refresh re-fetches current status.

1.7 Events / Schedule Screen

Purpose: Shows the athlete's registered events with live status.
Elements: List of events, each with heat, lane, reporting time, event time, and a status chip (Upcoming / Reporting / Running / Completed).
Buttons:

- **Event card tap** → expands to show call-room location and a countdown timer to reporting time. No write action — this is populated by push notifications and polling, not participant-triggered.

1.8 Results Screen

Purpose: Shows the athlete's own results once published.
Elements: Per-event card: time/score, position, qualification status ("Qualified for Final" / "Eliminated").
Buttons: None — populated automatically via the WebSocket results channel once an Event Head approves and publishes (see Section 5 of the architecture). Pull-to-refresh as a fallback if the live connection dropped.

1.9 Notifications Screen

Purpose: History of all push notifications received (reporting reminders, results, hostel/food alerts).
Elements: Reverse-chronological list, unread indicator.
Buttons:

- **Notification tap** → marks as read (`PATCH /notifications/:id/read`) and deep-links to the relevant screen (e.g., a "reporting in 15 min" notification opens the Events screen).

1.10 Maps Screen

Purpose: Venue orientation — hostels, food courts, event grounds, call rooms.
Elements: Map view with labeled pins, category filter chips.
Buttons:

- **Filter chips** (Hostels/Food/Events/Call Rooms) → client-side filter of visible pins, no API call.
- **Pin tap** → shows name + "Get Directions," which opens the device's native maps app.

---

2. COACH

2.1 Dashboard

Purpose: Oversight of every athlete the coach is responsible for.
Elements: Athlete list (name, college, current status), filter bar (Event / Gender /

College), search box.
Buttons:

- **Filter selections** → client-side filter of the already-fetched athlete list (list is scoped server-side to this coach's athletes at load time via `GET /coach/athletes`).
- **Search box** → filters by name/chest number as-you-type.
- **Athlete row tap** → opens Athlete Detail (2.2).

2.2 Athlete Detail Screen

Purpose: Single-athlete view for the coach.
Elements: Current location/status, upcoming event with countdown, hostel info, results history.
Buttons: All read-only — no write actions available to a coach anywhere in the system (coach capability is `view_reports` only, per the permission model). Pull-to-refresh re-fetches.

2.3 Notifications Screen

Purpose: Coach-relevant alerts: "Athlete X reporting in 15 min," "Athlete X qualified," "Athlete X marked absent."
Elements/Buttons: Same pattern as Participant 1.9 — tap marks read and deep-links to Athlete Detail.

---

3. REFEREE / OFFICIAL

3.1 Login Screen

Same OTP pattern as Participant (1.1), routes to Referee Home on success based on role in the JWT.

3.2 Home Screen

Purpose: Launchpad for the referee's event-day duties.
Elements: 4 cards — Scan Athlete, Call Room, Event Attendance, Results Entry — plus the referee's assigned event(s) shown at the top.

3.3 Scan Athlete Screen

Purpose: General-purpose QR scan/verify, used to confirm identity in any context that needs it.

Elements: Camera viewfinder, manual chest-number entry fallback (for a damaged/unreadable QR).
Buttons:

- **Camera scan** → decodes QR locally, validates signature against the cached roster (offline-capable per Section 4 of the architecture), shows athlete name/photo/event for the referee to visually confirm. No DB write from this screen alone — it's a lookup, the write happens in Call Room or Results Entry.
- **Manual entry** → same lookup path keyed by chest number instead of QR, for when a QR is damaged.

3.4 Call Room Screen

Purpose: Mark athletes present/absent before their heat.
Elements: List of athletes expected for the current heat, each with a Scan/Mark button.
Buttons:

- **Scan (per athlete)** → opens the camera scanner; on successful scan, the system cross-checks: correct athlete, correct event, correct heat. All three must match or it's rejected with a specific error ("This athlete is registered for Heat 3, not Heat 2").
- **Mark Present** (auto-set on successful scan match, or manually tappable as a fallback) → writes to `heats` attendance state.
- **Mark Absent** (manual, for athletes who don't show) → writes absent status; triggers a notification to the athlete's coach (per architecture Notification module).

3.5 Event Attendance Screen

Purpose: Roster-level view of who's checked into the current event across all heats.
Elements: Heat-grouped list with present/absent counts.
Buttons:

- **Heat row tap** → expands to the athlete list for that heat (read view, links back to Call Room to make changes).

3.6 Results Entry Screen

Purpose: Manual score/time entry for non-photo-finish events (field events, etc.).
Elements: Athlete list for the event, input field per athlete (time or distance

depending on event type), "Submit" button.
Buttons:

- **Submit** → validates all entries are present and plausible (non-negative, within a reasonable range for the event type), calls `POST /results/manual-entry`. System auto-computes ranking from the entered values and writes to `results` (staged, not yet published — matches the Event Head approval gate in the architecture). Confirmation dialog shown before submit ("This will submit results for 12 athletes — confirm?") since it's a one-shot action per heat.

---

4. EVENT HEAD

4.1 Home Screen

Purpose: Control center for the events this Event Head owns.
Elements: List of owned events, each showing current status (Not Started / In Progress / Locked / Published).

4.2 Heat Generation Screen

Purpose: Auto-generate heats and lane assignments.
Elements: Athlete count (auto-filled from registrations), lanes-per-heat input, "Generate Heats" button, preview table.
Buttons:

- **Generate Heats** → calls `POST /events/:id/generate-heats`, server runs the seeding algorithm (typically snake-seeded by qualifying time if available, otherwise random), returns a preview. Nothing is committed yet.
- **Confirm & Save** (appears after preview) → commits the generated heats/lanes to the DB. Regenerating after this point requires an explicit "Regenerate — this will clear existing heats" confirmation, since athletes may already have been notified.

4.3 Event Control Screen

Purpose: State machine control for the event's lifecycle.
Elements: Current state indicator, action buttons matching valid next states only.
Buttons:

- **Start Event** → transitions state to "In Progress," triggers the reporting-time notification sequence (30/15/10 min alerts) via the Notification module.

- **Lock Event** → freezes further heat/lane changes and closes Results Entry for this event to non-Event-Head roles. Irreversible without Super Admin `override_actions`.
- **Approve Results** → the human gate described in the architecture's photo-finish and results flow. Reviews staged results (from either manual entry or photo-finish import), can edit an individual value before approving, then publishes. Publishing pushes to the WebSocket results channel and triggers athlete/coach notifications.

---

5. HOSTEL WARDEN

5.1 Home Screen

Purpose: Launchpad — Scan QR, Check In, Check Out, Search Athlete.

5.2 Scan / Check-In / Check-Out Screen

Purpose: The actual hostel movement log.
Elements: Camera viewfinder, athlete result card (name, college, assigned hostel/room) after scan, Check In / Check Out buttons.
Buttons:

- **Scan** → decodes QR **locally against the cached roster** (offline-capable). If the scanned athlete's assigned hostel doesn't match this warden's hostel, shows a **red "Not Assigned Here"** alert immediately — this is a validation the original design correctly specified and it's preserved here.
- **Check In** → writes to `hostel_checkins` (local queue first, per the offline-sync design — Section 4 of the architecture), also updates the participant's Arrival Tracking status (1.4) to "Checked Into Hostel" automatically.
- **Check Out** → writes a paired checkout record; used for daily out-for-event / back-for-night tracking, not a one-time action.

5.3 Search Athlete Screen

Purpose: Manual lookup fallback when a QR can't be scanned.
Elements: Search box (name/chest number/college), result list.
Buttons:

- **Search result tap** → opens the same result card as a successful scan (5.2), same Check In/Out buttons available from there.

---

6. FOOD OFFICIAL

6.1 Home Screen

Purpose: 4 meal-specific scan stations — Breakfast, Lunch, Snacks, Dinner.

6.2 Meal Scan Screen (one per meal type, identical pattern)

Purpose: Prevent double-serving while keeping the line moving.
Elements: Camera viewfinder, result indicator (green check / red "Already Consumed").
Buttons:

- **Scan** → validates locally against cached roster + local "already consumed today" flag first (works with zero connectivity, per Section 4 of the architecture). If already consumed: red "Meal Already Consumed at [time]" — clearly loud enough that a busy counter operator can't miss it and wave the athlete through anyway. If valid: green confirmation, writes to local queue, syncs to `meal_transactions` when connectivity allows.

---

7. PHOTO-FINISH OPERATOR

7.1 Import Result Screen

Purpose: Bring timing-hardware output into the system without manual retyping.
Elements: Import method selector (CSV / XML / Direct Feed — depends on the actual hardware identified in Architecture Section 5), file picker or "Connect Device" button, preview table showing parsed lane/time pairs before submission.
Buttons:

- **Import method selector** → determines which adapter is used (per the adapter pattern in Architecture Section 5); this list is finalized only after Step 0 (identifying real hardware) is done.
- **File Picker / Connect Device** → loads raw data, runs it through the adapter to normalize into `{event_id, lane, time, sequence}`, shows a preview — nothing written to the DB yet.
- **Submit for Approval** → writes to the `photo_finish_imports` staging table (not `results` directly — this operator does **not** have `approve_results`, only `enter_results`, per the permission model). Notifies the relevant Event Head that results are ready for review.

---

8. ADMIN

8.1 Dashboard

Purpose: Real-time operational snapshot across the whole event.
Elements: Cards — Participants, Checked In, Meals Served, Events Running, Results Pending, Hostel Occupancy. All numbers pulled from the **read replica** (per Architecture Section 9) so this dashboard never slows down live scan traffic.
Buttons:

- **Card tap** (any) → drills into the relevant filtered list/report (e.g., "Meals Served" → Reports screen pre-filtered to today's meal transactions).

8.2 Reports Screen

Purpose: Cross-cutting views for operational decisions.
Elements: Filter tabs (By Event / By College / By Gender / By Hostel / By Coach), data table.
Buttons:

- **Filter tab** → re-queries the read replica with the selected grouping.
- **Export (Excel / CSV / PDF)** → calls `POST /reports/export`, generates the file server-side (worker job, per Architecture Section 2's background jobs), stores it in object storage, returns a download link — generation happens off the request thread so a big export doesn't block the API instance.

8.3 Sync Conflicts Screen

Purpose: Resolve the conflict cases flagged by the offline-sync reconciliation (Architecture Section 4).
Elements: List of flagged events (e.g., duplicate meal scans from two counters), each with both conflicting records shown side by side.
Buttons:

- **Resolve — Accept First / Accept Second / Reject Both** → writes the admin's decision as the authoritative record; the append-only log means the original conflicting entries are never deleted, just superseded, preserving the audit trail.

8.4 Hostel Allocation Upload Screen

Purpose: Bulk-set hostel capacity and trigger allocation.
Elements: Upload field (hostel name + capacity, e.g. "Hostel A = 200"), "Run

Allocation" button, preview.
Buttons:

- **Run Allocation** → runs the allocation algorithm (by college + gender grouping, per the original spec) against unassigned athletes, shows a preview of hostel/room/bed assignments before committing.
- **Confirm** → commits assignments, triggers a notification to every newly-assigned athlete with their hostel/room/bed and a map link.

8.5 User Management Screen

Purpose: Manage non-Super-Admin accounts.
Elements: User list, role badge per user, "Edit Role" action.
Buttons:

- **Edit Role** → opens a capability-bundle editor (per Architecture Section 8) — Admin can reassign a user to any existing role bundle, but cannot create new roles or grant `override_actions` (Super Admin only).

---

9. SUPER ADMIN

Has every Admin screen above, plus:

9.1 Role & Capability Management Screen

Purpose: The only place new roles get created or existing capability bundles get edited.
Elements: Capability checklist per role, "Create New Role" button.
Buttons:

- **Create New Role** → names a new role, checkbox-selects which capabilities it bundles (from the fixed capability list in Architecture Section 8), saves as a new `role_capabilities` row set.
- **Edit existing role's capabilities** → same checklist, changes apply to every user currently holding that role immediately (with a confirmation showing "This affects 4 users").

9.2 Override Actions Screen

Purpose: Manual overrides for situations the normal flow can't handle (broken scanner, disputed result, no-show correction).
Elements: Search-and-select athlete/event, override type selector (Force Check-In

/ Override Result / Unlock Event / Delete Record), reason text field (mandatory).
Buttons:

- **Force Check-In** → manually writes a hostel/event check-in record on an athlete's behalf, bypassing the scan requirement. Reason field is mandatory and stored — this is the highest-risk action in the system and the audit log (Architecture Section 11) exists specifically for this button.
- **Override Result** → edits a published result directly (normally results only change through the Event Head approval flow) — requires reason, logged, triggers a re-publish to the WebSocket channel and a "result corrected" notification to the affected athlete.
- **Unlock Event** → reverses an Event Head's "Lock Event" action (4.3), re-enabling heat/lane changes.
- **Delete Record** → soft-deletes (never hard-deletes, preserving the audit trail) — requires typing the record ID to confirm, the one double-confirmation pattern in the entire app.

End of document.